

LOW CODE EXPERIMENT 3

M.AMRUTHA VALLI

240701037

SURVEY REPORT ON EMERGING TRENDS IN LOW CODE APPLICATION

AIM

To study the emerging trends in low-code application development, understand how low-code platforms differ from traditional application development, and analyze their benefits, challenges, and commonly used platforms.

TRADITIONAL APPLICATION DEVELOPMENT VS. EMERGING TRENDS IN LOW-CODE APPLICATION DEVELOPMENT

Traditional Application Development

Traditional application development involves writing a large amount of program code manually. Developers generally use programming languages such as Java, Python, C++, JavaScript, or C#. The development process includes designing the application, writing code, testing, debugging, deploying, and maintaining the software.

This approach provides developers with greater control and flexibility. However, it can require more time, programming knowledge, and development resources.

Low-Code Application Development

Low-code application development uses visual development tools, drag-and-drop components, pre-built templates, workflows, and reusable modules to create applications with little manual coding. It allows developers and non-professional developers, often called *citizen developers*, to participate in application development.

Major Emerging Trends

1. AI-assisted development – Artificial intelligence is increasingly being used to generate application components, workflows, and code from natural-language instructions.
2. Drag-and-drop development – Visual interfaces allow users to build applications without writing every component manually.
3. Reusable components – Developers can reuse templates, APIs, modules, and workflows.
4. Cloud-based development – Many low-code platforms provide cloud-based development, deployment, and collaboration.
5. Mobile application development – Low-code tools can be used to develop applications for web and mobile devices.
6. Automation and workflow integration – Platforms provide tools for automating repetitive business processes.
7. API and database integration – Low-code applications can connect with databases, APIs, and external services.
8. Citizen development – People with limited programming experience can create applications with guidance from IT teams.
9. Security and governance – Modern platforms increasingly provide authentication, access control, monitoring, and governance features.

Difference Between Traditional and Low-Code Development

Feature	Traditional Development	Low-Code Development
Coding	Extensive manual coding	Minimal coding
Development method	Code-based	Visual and model-based
Programming knowledge	Usually required	Basic knowledge may be sufficient
Development speed	Comparatively slower	Generally faster
Customization	Highly flexible	Depends on platform
Components	Developed manually or through libraries	Many pre-built components
Testing	Mostly developer-managed	Often includes built-in tools
Deployment	Usually requires technical setup	Often simplified through the platform
Target users	Professional developers	Developers and citizen developers
Maintenance	Mostly code-based	Platform-supported and component-based

BENEFITS AND CHALLENGES OF LOW-CODE DEVELOPMENT

Benefits

1. Faster Development:

Applications can be created more quickly using drag-and-drop components and ready-made templates.

2. Reduced Development Effort:

Developers do not have to write every part of an application from scratch.

3. Easy to Learn:

The visual interface makes application development easier for beginners and students.

4. Increased Productivity:

Developers can concentrate on application logic and business requirements instead of repetitive coding tasks.

5. Rapid Prototyping:

A working prototype can be created quickly and demonstrated to users.

6. Automation:

Low-code platforms can automate workflows and repetitive business processes.

7. Integration:

Many platforms provide connectors and APIs for connecting applications with databases and external services.

Challenges

1. Limited Customization:

Some complex requirements may not be easy to implement using only the features provided by a platform.

2. Vendor Lock-in:

Applications can become dependent on a particular platform, making migration difficult.

3. Security Concerns:

Applications must be configured correctly to protect data and prevent unauthorized access.

4. Performance Limitations:

For highly complex or resource-intensive applications, a traditional development approach may provide greater control over performance.

5. Platform Dependency:

Changes in pricing, features, or policies of the platform provider can affect applications.

4. Comparison of Existing Low-Code Platforms

Platform	Main Purpose	Key Features	Suitable For
Microsoft Power Apps	Business apps	Visual builder, connectors, Dataverse	Organizations
OutSystems	Enterprise apps	Visual development, integrations, deployment	Enterprises
Mendix	App development	Visual modeling, collaboration	Businesses
Appian	Process automation	Workflows, process management	Enterprise processes
Salesforce	CRM & business apps	Automation, CRM integration	CRM applications
Google AppSheet	No-code apps	Data apps, automation, Google integration	Small businesses
Zoho Creator	Custom business apps	Forms, workflows, reports	SMEs

5. RESULT

The survey shows that low-code application development is an emerging approach that simplifies and accelerates software development by using visual interfaces, pre-built components, automation, and reusable modules.

Compared with traditional application development, low-code platforms can reduce manual coding and make application development accessible to a wider group of users. Current trends such as AI-assisted development, cloud-based development, workflow automation, API integration, reusable components, and citizen development are increasing the capabilities of low-code platforms.